

Video Script & Protocol

Episode 3: Autonomous Code Generator & Visual Validator (qwen2.5v1)

Channel: Data, Paper & Code

September 30, 2026

Contents

1	Structure & Outline	2
2	YouTube Speaker Script	2
3	Theoretical Background	3
3.1	Why Self-Corrective Agent Loops?	3
3.2	Agent Loop Architecture	4
4	System Setup & Prerequisites	4
5	Complete Python Script of the Agent	4
6	Results & Agent Execution Protocol	8
7	Outlook & Application Areas	8

1 Structure & Outline

No.	Topic Block	Core Content & Objective
01	Greeting & Hook	The limits of blind coding LLMs.
02	The Agent Pattern	Closed-loop feedback: Generate, execute, visually validate.
03	Subprocess Execution	Safe execution of LLM code in isolated runtimes.
04	Vision Feedback Loop	How <code>qwen2.5v1</code> reads charts and grants approval (APPROVED).
05	Live Demo & Execution	Step-by-step run of the Python agent.
06	Summary & Outlook	Applications for automated reports & dashboards.

Table 1: Schedule for the third video episode

2 YouTube Speaker Script

Part 1: The Hook — When LLMs Blindly Write Code

[Camera: Close-up | Dynamic & direct]

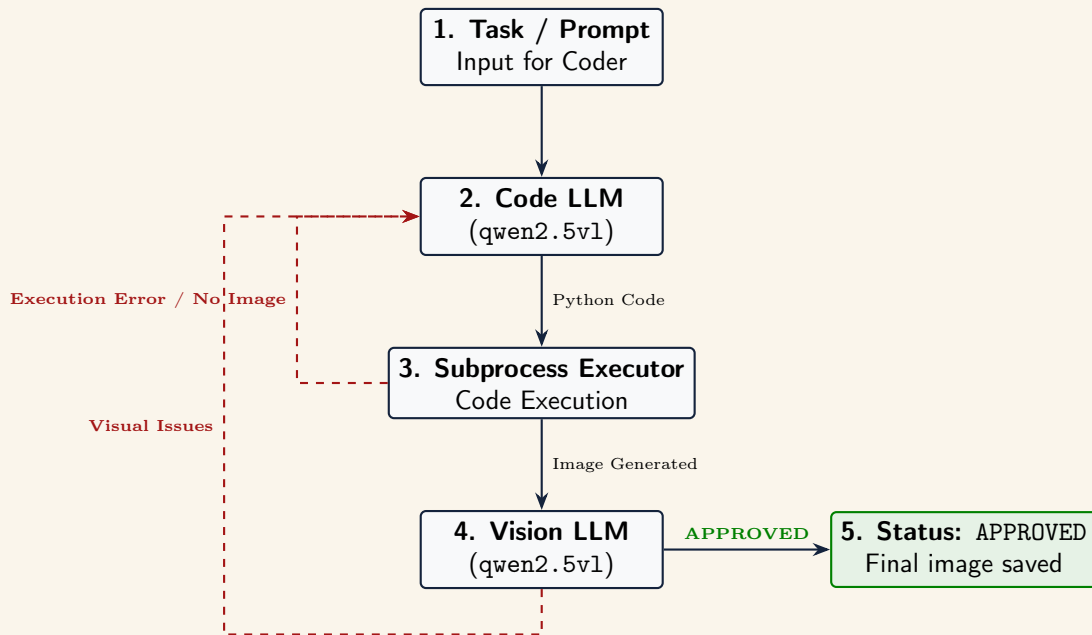
"Imagine asking a language model to draw a nice financial chart for you. The model dutifully delivers the Python code, you run it — and BOOM: Either the script crashes with a massive traceback, or the graphic looks terrible. Overlapping axis labels, missing titles, or truncated data.

The problem? Classic coding LLMs work completely ****blind****. They have no idea what their generated code actually looks like visually.

Time to fix that. Welcome to **Data, Paper & Code**! Today, we are building a ****fully autonomous, local software agent****. It writes Python code, executes it locally, catches errors, and leverages the local vision model `qwen2.5v1` via Ollama to visually verify the generated chart. Only when the vision model responds with **APPROVED** is the job done. Completely local, free, and without cloud APIs!"

Part 2: The Agent Pattern (Closed-Loop Loop)

[Screen: Architecture diagram of the iteration loop]



"Our agent uses a clever looping mechanism: 1. **The Coder:** `qwen2.5v1` generates pure Python code based on our prompt. 2. **The Executor:** A Python `subprocess` executes the code. If a syntax or runtime error occurs, the error log goes straight back to the coder for self-correction. 3. **The Validator:** Once the script successfully produces an image, the vision LLM steps in. It evaluates the image for legibility, axis labels, and overall layout. If the vision model gives the green light with **APPROVED**, the loop stops. If not, the critique is passed back for the next iteration!"

Part 3: Live Demo & The Perfect Run

[Screen: VS Code / Terminal open]

"Let's watch our agent work live! Our task for the agent: Download stock data using `yfinance` for the ETF `VUSD.L` and plot a neatly formatted chart. Take a look at the console: In **Iteration 1**, the coder generates the Python code. The subprocess executes it and produces the file `Curve_iter_1.jpg`. Now, our agent passes the generated image to Ollama. `qwen2.5v1` inspects the image, verifies axes, grid lines, and titles... and BOOM! The vision model directly replies with **APPROVED**. The agent saves the final result as `Curve_final.jpg` and completes the task on the very first try!"

3 Theoretical Background

3.1 Why Self-Corrective Agent Loops?

Classic code generation suffers from the *open-loop problem*: errors are only discovered after manual execution by a developer. A self-corrective agent closes this gap through automated traceback parsing and visual feedback.

3.2 Agent Loop Architecture

The system architecture splits into three functional components:

1. **Code Generation Phase:** Generation of isolated Python code without markdown tags, controlled via ChatML system prompts.
2. **Subprocess Sandbox Execution:** Execution of the generated script via the Python interpreter using `subprocess.run`. Runtime errors are captured via `stderr`.
3. **Multimodal Visual Validation:** Embedding the result as a Base64 string into the Ollama API. A specially structured prompt requests a binary decision (APPROVED vs. refinement request).

4 System Setup & Prerequisites

Instructions: Preparing the Agent Environment

1. **Set up Ollama:** Make sure Ollama is running in the background.
2. **Pull the Model:** Download the multimodal model once:

```
ollama run qwen2.5vl
```

3. **Install Python Packages:**

```
pip install requests yfinance matplotlib pillow
```

5 Complete Python Script of the Agent

The following script manages the complete autonomous verification and correction process.

```
1 import base64
2 import os
3 import subprocess
4 import sys
5 import requests
6
7 OLLAMA_GENERATE_URL = "http://localhost:11434/api/generate"
8 CODER_MODEL = "qwen2.5vl"
9 VISION_MODEL = "qwen2.5vl"
10
11 SCRIPT_FILE = "generated_chart_script.py"
12 TEMPLATE_IMAGE = "Curve_Template.jpg"
13 BASE_IMAGE_NAME = "Curve"
14 MAX_ATTEMPTS = 4
15
16
17 def encode_image_to_base64(image_path: str) -> str:
18     """Converts a local image file to a base64 string."""
19     with open(image_path, "rb") as image_file:
```

```

20         return base64.b64encode(image_file.read()).decode('utf-8')
21
22
23 def analyze_chart_with_ollama(
24     image_path: str,
25     prompt_text: str,
26     ollama_url: str = OLLAMA_GENERATE_URL,
27     model_name: str = VISION_MODEL
28 ) -> str:
29     """Sends the image to the Vision model in Ollama."""
30     if not os.path.exists(image_path):
31         return "[ERROR] The image was not created."
32
33     base64_image = encode_image_to_base64(image_path)
34
35     payload = {
36         "model": model_name,
37         "prompt": prompt_text,
38         "images": [base64_image],
39         "stream": False,
40         "options": {"temperature": 0.2}
41     }
42
43     try:
44         response = requests.post(ollama_url, json=payload, timeout=120)
45         response.raise_for_status()
46         return response.json().get("response", "No response received from
the model.")
47     except requests.exceptions.RequestException as e:
48         return f"[ERROR]: Request to Ollama failed: {e}"
49
50
51 def run_code_llm(prompt: str) -> str:
52     """Queries the Coder model via Ollama."""
53     full_prompt = (
54         "<|im_start|>system\n"
55         "You are an expert Python developer. Write complete, valid Python
code.\n"
56         "Do NOT use markdown backticks (no ``python). Return ONLY the
executable Python code.<|im_end|>\n"
57         f"<|im_start|>user\n{prompt}<|im_end|>\n"
58         "<|im_start|>assistant\n"
59     )
60
61     payload = {
62         "model": CODER_MODEL,
63         "prompt": full_prompt,
64         "stream": False,
65         "options": {
66             "temperature": 0.2,
67             "num_predict": 1024
68         }
69     }
70
71     try:
72         response = requests.post(OLLAMA_GENERATE_URL, json=payload,
timeout=90)
73         response.raise_for_status()
74

```

```

75     result_json = response.json()
76     code_output = result_json.get("response", "").strip()
77
78     if not code_output:
79         payload["prompt"] = prompt
80         res = requests.post(OLLAMA_GENERATE_URL, json=payload, timeout
=90).json()
81         code_output = res.get("response", "").strip()
82
83     return code_output
84
85 except Exception as e:
86     print(f"      Error requesting Coder model ({CODER_MODEL}): {e}")
87     return ""
88
89
90 def run_agent(task_description_en: str):
91     print("=====")
92     print("AGENT: Chart Code Generator & Visual Validator")
93     print(f"Template: {TEMPLATE_IMAGE}")
94     print("=====\\n")
95
96     history = ""
97
98     for attempt in range(1, MAX_ATTEMPTS + 1):
99         print(f"--- Iteration {attempt}/{MAX_ATTEMPTS} ---")
100
101         current_image_file = f"{BASE_IMAGE_NAME}_iter_{attempt}.jpg"
102
103         # 1. GENERATE CODE
104         prompt = (
105             f"You are a Python developer. Task: {task_description_en}\\n"
106             f"Requirements:\\n"
107             f"- Save the generated chart image strictly as '{
current_image_file}'.\\n"
108             f"- Return ONLY the raw executable Python code. No markdown
blocks, no conversational text.\\n"
109             f"{history}"
110         )
111
112         raw_code = run_code_llm(prompt)
113
114         if not raw_code or len(raw_code.strip()) == 0:
115             print("      Error: The LLM returned an empty string. Retrying
...")
116             history += "\\nYour last response was completely empty. You
MUST generate valid Python code!"
117             continue
118
119         # Clean markdown tags
120         code = raw_code
121         if code.startswith("```"):
122             lines = code.split("\\n")
123             code = "\\n".join(lines[1:-1]) if lines[-1].startswith("```")
else "\\n".join(lines[1:])
124
125         with open(SCRIPT_FILE, "w", encoding="utf-8") as f:
126             f.write(code)
127

```

```

128     # 2. EXECUTE CODE
129     res = subprocess.run([sys.executable, SCRIPT_FILE], capture_output
130                             =True, text=True)
131
132     if res.returncode != 0:
133         print(f"    Syntax/Runtime Error:\n{res.stderr.strip()}")
134         history += f"\nPrevious attempt failed with error:\n{res.
135             stderr}\nFix the code."
136         continue
137
138     if not os.path.exists(current_image_file):
139         print(f"    Error: The script executed successfully, but
140             failed to generate '{current_image_file}'")
141         history += f"\nThe script ran, but failed to save the image to
142             '{current_image_file}'. Make sure to call plt.savefig('{
143             current_image_file}')."
144         continue
145
146     print(f"    Script executed successfully and created '{
147         current_image_file}'")
148
149     # 3. VISION ANALYSIS
150     vision_prompt = (
151         "Analyze the generated chart picture critically:\n"
152         "1. Is the chart correctly rendered and legible?\n"
153         "2. Are axes labeled properly without overlapping text?\n"
154         "If the chart looks complete and well-formatted, reply with '
155         APPROVED'. "
156         "Otherwise, explain shortly what needs to be improved."
157     )
158
159     print(f"[INFO] Sending '{current_image_file}' for visual
160         validation to {VISION_MODEL}...")
161
162     evaluation = analyze_chart_with_ollama(current_image_file,
163         vision_prompt)
164     print(f"Vision Feedback:\n{evaluation}\n")
165
166     if "APPROVED" in evaluation.upper():
167         print("    SUCCESS: The script generated an error-free and
168             visually verified chart!\n")
169         print("=" * 50)
170         print("    CONTENT OF generated_chart_script.py:")
171         print("=" * 50)
172         print(code)
173
174         final_image = f"{BASE_IMAGE_NAME}_final.jpg"
175         with open(current_image_file, "rb") as src, open(final_image,
176             "wb") as dst:
177             dst.write(src.read())
178         print(f"    The final chart was saved as '{final_image
179             }'")
180         break
181
182     history += f"\nThe generated image had visual issues:\n{evaluation
183         }\nPlease update the code."
184
185 if __name__ == "__main__":

```

```

174     task = (
175         "Write a Python script using yfinance and matplotlib to download 3
176         months of data for 'VUSD.L'. "
177         "Plot the Close price with a grid and proper title."
178     )
    run_agent(task)

```

Listing 1: Complete Python Code for the Vision Code Agent

6 Results & Agent Execution Protocol

Protocol of the Successful Agent Run

Execution: python Software_Agent03_with_image.py

=====

AGENT: Chart Code Generator & Visual Validator

=====

-- Iteration 1/4 --

Script executed successfully and created 'Curve_iter_1.jpg'.

[INFO] Sending 'Curve_iter_1.jpg' for visual validation to qwen2.5vl...

Vision Feedback:

APPROVED. The chart is correctly rendered and legible. The axes are labeled properly without overlapping text. The title and the legend are clearly visible and the data points are distinct.

Result: SUCCESS: The script generated an error-free and visually verified chart!

7 Outlook & Application Areas

The self-corrective agent pattern demonstrated here bridges the gap between generative AI and rigorous scientific data analysis. It translates seamlessly into automated pipelines across research and development:

- **Scientific Regression Diagnostics:** Automatic generation, visual verification, and interpretation of complex diagnostic plots (e.g., *Normalized Residual Plots* for checking model assumptions).
- **Quality Assurance for Research Papers:** Automated compliance checks for generated figures regarding legibility, proper scaling, and journal formatting guidelines (e.g., IEEE or Nature style limits).
- **Autonomous Reports:** Dynamic generation of reproducible PDF reports from raw data pipelines with automated visual self-auditing of embedded charts.
- **Multimodal Benchmarking Systems:** Automated evaluation of open-source vision LLMs on their capability to interpret technical and mathematical figures accurately.